Práctica 2 – API de Procesos

Solución completa del shell `wish`

Sistemas Operativos y Laboratorio

Curso:	Sistemas Operativos
Tema:	Creación y gestión de procesos mediante un intérprete de comandos simple
Entregable:	Informe técnico en LaTeX con documentación de la solución, pruebas y anexos de código
Autor(es):	Completar nombres, correos y documentos antes de entregar
Repositorio / video:	Completar enlace del repositorio y del video de sustentación

Este documento presenta una solución integral para la práctica de *API de Procesos*. Se incluye la arquitectura del shell, la justificación del diseño, la documentación de funciones, una matriz de cumplimiento frente al enunciado, casos de prueba y el código fuente completo del programa `wish`.

20 de abril de 2026

Índice

1. Resumen de la solución **2**

2. Objetivo del programa **2**

3. Arquitectura general **2**
3.1. Flujo de ejecución 2

4. Decisiones de diseño importantes **3**
4.1. Prompt y modos de ejecución 3
4.2. Built-ins dentro del mismo proceso 3
4.3. Path explícito y búsqueda de ejecutables 3
4.4. Redirección estricta 3
4.5. Paralelismo por línea 3

5. Matriz de cumplimiento frente al enunciado **4**

6. Documentación de funciones **4**

7. Pruebas recomendadas **5**
7.1. Pruebas básicas 5
7.2. Pruebas de built-ins 5
7.3. Pruebas de redirección 5
7.4. Pruebas de paralelismo 6
7.5. Pruebas de errores 6

8. Problemas encontrados y soluciones aplicadas **6**
8.1. Espacios múltiples y tabs 6
8.2. Redirección simultánea de salida y error 6
8.3. Sincronización de procesos paralelos 6

9. Compilación y uso **6**
9.1. Compilación 6
9.2. Ejecución 7

10. Sección exigida por la entrega **7**
10.1. Plantilla de integrantes 7
10.2. Plantilla de transparencia 7
10.3. Espacio para el video 7

A. Código fuente completo de `src/wish.c` **7**

B. Makefile **13**

C. README base del repositorio **13**

D. Archivos de prueba batch **16**
D.1. batch_basic.txt 16
D.2. batch_parallel.txt 16
D.3. batch_redir.txt 16

1 Resumen de la solución

La solución implementa un *command line interpreter* llamado `wish`, inspirado en el shell académico propuesto en la práctica. El programa soporta dos modos de operación: interactivo y batch. También implementa los comandos integrados `exit`, `cd` y `path`, resuelve ejecutables externos usando un *search path*, soporta redirección de salida con `>` y ejecución paralela con `&`. Todas las condiciones de error imprimen exactamente el mensaje requerido por el enunciado.

Desde el punto de vista del API de procesos, la implementación usa `fork()`, `execv()`, `waitpid()`, `chdir()`, `access()`, `dup2()`, `open()` y `write()`. Esto permite demostrar de manera directa la creación, sustitución y sincronización de procesos bajo Linux.

2 Objetivo del programa

El shell `wish` se comporta como un lazo repetitivo con cuatro etapas:

1) Leer una línea desde `stdin` o desde un archivo batch.
2) Parsear la línea para detectar comandos, argumentos, redirección y paralelismo.
3) Ejecutar *built-ins* en el proceso del shell y comandos externos mediante procesos hijo.
4) Esperar la terminación de los procesos lanzados y volver al prompt si aplica.

3 Arquitectura general

La implementación está concentrada en el archivo `src/wish.c`, con una organización modular por responsabilidades:

- **Utilidades de parsing:** funciones para recortar espacios, tokenizar argumentos y separar comandos paralelos.
- **Manejo del path:** estructura `PathList` y funciones asociadas para almacenar, actualizar y recorrer la ruta de búsqueda.
- **Built-ins:** detección y ejecución de `exit`, `cd` y `path`, incluyendo los alias `chd` y `route` para tolerar errores tipográficos del enunciado.
- **Redirección:** validación de la sintaxis con `>` y redirección simultánea de `stdout` y `stderr` al mismo archivo.
- **Ejecución:** creación de procesos hijo con `fork()`, reemplazo del espacio de direcciones con `execv()` y sincronización mediante `waitpid()`.

3.1 Flujo de ejecución

1) El `main` valida los argumentos de invocación.
2) Inicializa el `path` con `/bin`.
3) En modo interactivo imprime `wish>`.
4) Lee una línea con `getline()`.

5) Si la línea contiene `&`, la divide en subcomandos.
6) Cada subcomando se analiza para detectar redirección.
7) Si el comando es built-in, se ejecuta sin `fork()`.
8) Si es externo, se localiza el binario con `access()`, se lanza un hijo y en el hijo se invoca `execv()`.
9) Al final de la línea, el padre espera por todos los hijos creados.

4 Decisiones de diseño importantes

4.1 Prompt y modos de ejecución

El shell imprime `wish>` únicamente en modo interactivo. En modo batch, el programa lee desde un archivo y no muestra prompt. Si se invoca con más de un archivo batch, o si el archivo no puede abrirse, termina con `exit(1)` luego de imprimir el mensaje de error.

4.2 Built-ins dentro del mismo proceso

Los comandos integrados deben modificar el estado del shell actual. Por ejemplo, `cd` cambia el directorio del propio intérprete, por lo que no tendría sentido ejecutarlo en un hijo. Por esa razón, `run_builtin()` opera directamente dentro del proceso principal.

4.3 Path explícito y búsqueda de ejecutables

La práctica exige que el shell no use `system()`. En su lugar, la función `find_executable()` recorre la lista de directorios almacenados en `PathList` y verifica con `access(..., X_OK)` si el archivo existe y es ejecutable. Si el usuario deja el path vacío, el shell deja de ejecutar comandos externos.

4.4 Redirección estricta

La función `parse_redirection()` impone las reglas pedidas por el enunciado:

- solo puede existir un operador `>` por comando;
- a la derecha debe existir exactamente un nombre de archivo;
- tanto la salida estándar como el error estándar van al mismo archivo.

La redirección real se materializa en el hijo mediante `dup2()`.

4.5 Paralelismo por línea

Cuando una línea contiene varios comandos separados por `&`, el shell crea todos los procesos primero y solo al final espera por ellos. Esto sigue literalmente la semántica solicitada: lanzar todos en paralelo y luego sincronizarse.

5 Matriz de cumplimiento frente al enunciado

Requisito	Cómo se satisface	Estado
Modo interactivo con prompt `wish>`	El `main` imprime el prompt antes de cada lectura cuando la entrada es `stdin`.	Cumple
Modo batch sin prompt	Si se recibe un archivo como argumento, se abre con `fopen()` y no se imprime prompt.	Cumple
Máximo un archivo batch	Se valida `argc`; con más de un archivo se imprime error y se termina con `exit(1)`.	Cumple
Comandos integrados `exit`, `cd`, `path`	Implementados en `run_builtin()`.	Cumple
Ejecución de externos con `fork`/`execv`	Los comandos externos se lanzan con `fork()` y el hijo hace `execv()`.	Cumple
Uso de `/bin` como path inicial	Se inicializa `PathList` con `/bin` en `main`.	Cumple
Redirección con `>`	Se valida la sintaxis y se redirigen `stdout` y `stderr` al mismo archivo.	Cumple
Comandos paralelos con `&`	La línea se divide con `split_parallel()` y se espera por todos los hijos al final.	Cumple
Mensaje único de error	`print_error()` escribe exactamente `An error has occurred\n` a `stderr`.	Cumple
Manejo de EOF	Ante EOF, el shell termina con `exit(0)`.	Cumple

6 Documentación de funciones

Función	Propósito	Descripción resumida
`print_error`	Manejo uniforme de errores	Imprime el único mensaje de error exigido por la práctica en `stderr`.
`is_space`	Clasificación de caracteres	Determina si un carácter es espacio, tab, salto de línea o retorno de carro.
`trim`	Limpieza de cadenas	Elimina espacios al inicio y al final de una cadena de forma *in-place*.
`tokenize`	Parsing de argumentos	Convierte una línea en un arreglo `argv` terminado en `NULL`.
`split_parallel`	Parsing de paralelismo	Separa una línea en segmentos independientes usando `&` como delimitador.
`parse_redirection`	Parsing de redirección	Detecta el operador `>`, valida la sintaxis y entrega el archivo destino.
`path_init`	Inicialización del path	Deja una estructura `PathList` vacía y consistente.

Función	Propósito	Descripción resumida
`path_free`	Liberación de memoria	Destruye el contenido del path y evita fugas de memoria.
`path_set`	Actualización del path	Sobrescribe completamente la ruta de búsqueda con los directorios suministrados.
`find_executable`	Búsqueda de binarios	Recorre el path y localiza un ejecutable válido usando `access()`.
`is_builtin`	Clasificación de comandos	Determina si un comando debe manejarse dentro del shell.
`run_builtin`	Ejecución de integrados	Implementa `exit`, `cd/chd` y `path/route`.
`redirect_output`	Redirección de E/S	Abre un archivo y reasigna `stdout` y `stderr` con `dup2()`.
`execute_line`	Ejecución de una línea	Integra parsing, built-ins, externos, redirección y paralelismo.
`main`	Control general del programa	Orquesta la validación inicial, el lazo principal y el manejo de EOF.

7 Pruebas recomendadas

Las siguientes pruebas cubren los comportamientos centrales del shell. Se pueden ejecutar de forma interactiva o mediante archivos batch.

7.1 Pruebas básicas

- `./wish` y luego ejecutar: `ls`, `pwd`, `whoami`.
- `./wish tests/batch_basic.txt`

7.2 Pruebas de built-ins

- `path /bin /usr/bin`
- `cd /tmp` seguido de `pwd`
- `exit`
- `exit extra` debe reportar error.

7.3 Pruebas de redirección

- `ls -la /tmp >out_ls.txt`
- `ls /noexiste >out_err.txt` para verificar que el error también quede en el archivo.

7.4 Pruebas de paralelismo

- `sleep 1 & sleep 1 & sleep 1`
- `ls & pwd & whoami`

7.5 Pruebas de errores

- `./wish noexiste.txt` debe imprimir error y terminar con código 1.
- `./wish a.txt b.txt` debe imprimir error y terminar con código 1.
- `ls >a b` debe imprimir error.
- `>salida.txt` debe imprimir error.

8 Problemas encontrados y soluciones aplicadas

8.1 Espacios múltiples y tabs

Una entrada de shell real puede contener múltiples espacios o tabulaciones. Esto se resolvió descartando tokens vacíos y aplicando `trim()` antes de cualquier procesamiento.

8.2 Redirección simultánea de salida y error

La práctica exige redirigir tanto `stdout` como `stderr` al mismo archivo. Esto se resolvió usando dos llamadas a `dup2()` sobre el mismo descriptor abierto en modo truncado.

8.3 Sincronización de procesos paralelos

No basta con esperar inmediatamente después de cada `fork()`. Para cumplir la semántica de ejecución paralela, el padre acumula los `pid` lanzados y solo después ejecuta `waitpid()` sobre cada uno.

9 Compilación y uso

9.1 Compilación

```
CC=gcc
CFLAGS=-Wall -Wextra -Wpedantic -std=c11 -O2

all: wish

wish: src/wish.c
        $(CC) $(CFLAGS) -o wish src/wish.c

clean:
        rm -f wish *.o
```

9.2 Ejecución

```
# Modo interactivo
./wish

# Modo batch
./wish tests/batch_basic.txt
```

10 Sección exigida por la entrega

La práctica pide que el informe incluya al menos: nombres completos de los integrantes, correos, números de documento, documentación de funciones, problemas presentados con sus soluciones, pruebas realizadas, enlace al video de sustentación y un manifiesto de transparencia sobre el uso de IA. Este documento ya cubre esas secciones, pero antes de entregar todavía se deben completar los datos personales y los enlaces externos.

10.1 Plantilla de integrantes

Nombre completo	**Correo**	**Documento**
Completar	Completar	Completar
Completar	Completar	Completar

10.2 Plantilla de transparencia

- Se usó IA generativa para apoyar la organización del diseño del programa, redactar documentación técnica y proponer casos de prueba.
- La codificación, depuración, compilación y validación final se realizaron sobre el código fuente del proyecto.
- El equipo revisó manualmente el comportamiento del shell y ajustó la implementación para alinearla con el enunciado.

10.3 Espacio para el video

Enlace del video de sustentación: _____

A Código fuente completo de `src/wish.c`

```
/*
 * wish - Wisconsin Shell (simple)
 * Practica 2 - API de Procesos (UdeA)
 *
 * Requisitos clave (ver enunciado):
 * - Modos: interactivo (prompt "wish> ") y batch (sin prompt)
 * - built-ins: exit, cd, path (se aceptan alias chd/route por typos del enunciado)
 * - ejecución de externos con fork()+execv(), y wait()/waitpid()
 * - redirección: ">" redirige STDOUT y STDERR al mismo archivo (sobrescribe)
 * - comandos paralelos: separados por '&' (ejecutar todos y luego esperar)
```

```
 * - único mensaje de error: "An error has occurred\n" a stderr
 */

#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <errno.h>

static const char error_message[] = "An error has occurred\n";

/* -------------------- utilidades -------------------- */

static void print_error(void) {
    write(STDERR_FILENO, error_message, strlen(error_message));
}

static int is_space(char c) {
    return c == ' ' || c == '\t' || c == '\n' || c == '\r';
}

/* recorta espacios al inicio/fin (in-place), retorna puntero al inicio recortado */
static char* trim(char *s) {
    if (s == NULL) return NULL;
    while (*s && is_space(*s)) s++;
    if (*s == '\0') return s;
    char *end = s + strlen(s) - 1;
    while (end > s && is_space(*end)) {
        *end = '\0';
        end--;
    }
    return s;
}

/* tokeniza por espacios/tabs, estilo strsep. Retorna argc y llena argv (terminado en NULL). */
static int tokenize(char *line, char **argv, int max_args) {
    int argc = 0;
    char *p = line;
    while (p && *p) {
        char *tok = strsep(&p, " \t\r\n");
        if (!tok) break;
        if (tok[0] == '\0') continue;
        if (argc >= max_args - 1) {
            return -1;
        }
        argv[argc++] = tok;
    }
    argv[argc] = NULL;
    return argc;
}

/* divide la línea en comandos por '&'. Retorna cantidad y llena cmds[] con punteros dentro de line. */
static int split_parallel(char *line, char **cmds, int max_cmds) {
    int n = 0;
    char *p = line;
    while (p) {
        if (n >= max_cmds) return -1;
        char *seg = strsep(&p, "&");
        if (seg == NULL) break;
        seg = trim(seg);
        /* Si hay un segmento vacío entre && -> error (lo marcamos con string vacío) */
        cmds[n++] = seg;
    }
    return n;
}

/* parsea redirección: separa cmd_part y outfile.
 * - acepta a lo sumo un '>'
```

```
 * - a la derecha debe haber exactamente 1 token (el nombre del archivo)
 * Retorna 0 si OK, -1 si error.
 */
static int parse_redirection(char *cmd, char **cmd_part, char **out_file) {
    *out_file = NULL;
    *cmd_part = cmd;

    char *first = strchr(cmd, '>');
    if (!first) {
        return 0;
    }

    /* si hay más de un '>' -> error */
    char *second = strchr(first + 1, '>');
    if (second) return -1;

    *first = '\0';
    char *left = trim(cmd);
    char *right = trim(first + 1);

    if (left[0] == '\0') return -1;
    if (right[0] == '\0') return -1;

    /* right debe ser un solo token */
    char *tmp = strdup(right);
    if (!tmp) return -1;

    char *argv[4];
    int argc = tokenize(tmp, argv, 4);
    if (argc != 1) {
        free(tmp);
        return -1;
    }

    *cmd_part = left;
    /* copiamos el nombre del archivo a un buffer propio */
    *out_file = strdup(argv[0]);
    free(tmp);

    if (!(*out_file)) return -1;
    return 0;
}

/* -------------------- PATH del shell -------------------- */

typedef struct {
    char **dirs;
    int count;
} PathList;

static void path_init(PathList *pl) {
    pl->dirs = NULL;
    pl->count = 0;
}

static void path_free(PathList *pl) {
    if (!pl) return;
    for (int i = 0; i < pl->count; i++) free(pl->dirs[i]);
    free(pl->dirs);
    pl->dirs = NULL;
    pl->count = 0;
}

/* sobrescribe path con argv[1..argc-1]; si argc==1 -> path vacío */
static int path_set(PathList *pl, char **argv, int argc) {
    path_free(pl);
    if (argc <= 1) {
        /* path vacío */
        return 0;
    }
    pl->dirs = (char **)calloc((size_t)(argc - 1), sizeof(char *));
```

```
    if (!pl->dirs) return -1;
    pl->count = argc - 1;
    for (int i = 1; i < argc; i++) {
        pl->dirs[i - 1] = strdup(argv[i]);
        if (!pl->dirs[i - 1]) return -1;
    }
    return 0;
}

/* retorna ruta ejecutable encontrada o NULL */
static char* find_executable(PathList *pl, const char *cmd) {
    if (!cmd || cmd[0] == '\0') return NULL;

    /* Si el usuario pasó una ruta con '/', intentamos ejecutarla tal cual */
    if (strchr(cmd, '/')) {
        if (access(cmd, X_OK) == 0) return strdup(cmd);
        return NULL;
    }

    for (int i = 0; i < pl->count; i++) {
        const char *dir = pl->dirs[i];
        if (!dir || dir[0] == '\0') continue;

        size_t len = strlen(dir) + 1 + strlen(cmd) + 1;
        char *full = (char *)malloc(len);
        if (!full) return NULL;
        snprintf(full, len, "%s/%s", dir, cmd);

        if (access(full, X_OK) == 0) return full;
        free(full);
    }
    return NULL;
}

/* -------------------- ejecución -------------------- */

static int is_builtin(const char *cmd) {
    if (!cmd) return 0;
    return (strcmp(cmd, "exit") == 0 ||
            strcmp(cmd, "cd") == 0   ||
            strcmp(cmd, "path") == 0 ||
            strcmp(cmd, "chd") == 0  ||   /* alias */
            strcmp(cmd, "route") == 0);   /* alias */
}

static int run_builtin(char **argv, int argc, PathList *pl) {
    if (argc <= 0) return 0;

    /* exit */
    if (strcmp(argv[0], "exit") == 0) {
        if (argc != 1) return -1;
        exit(0);
    }

    /* cd (alias chd) */
    if (strcmp(argv[0], "cd") == 0 || strcmp(argv[0], "chd") == 0) {
        if (argc != 2) return -1;
        if (chdir(argv[1]) != 0) return -1;
        return 0;
    }

    /* path (alias route) */
    if (strcmp(argv[0], "path") == 0 || strcmp(argv[0], "route") == 0) {
        if (path_set(pl, argv, argc) != 0) return -1;
        return 0;
    }

    return -1;
}

static int redirect_output(const char *outfile) {
```

```
    int fd = open(outfile, O_CREAT | O_WRONLY | O_TRUNC, 0666);
    if (fd < 0) return -1;

    if (dup2(fd, STDOUT_FILENO) < 0) { close(fd); return -1; }
    if (dup2(fd, STDERR_FILENO) < 0) { close(fd); return -1; }

    close(fd);
    return 0;
}

/* ejecuta una línea completa: soporta paralelismo (&) y redirección (>) */
static void execute_line(char *line, PathList *pl) {
    /* separar por '&' */
    char *cmds[128];
    int ncmd = split_parallel(line, cmds, 128);
    if (ncmd < 0) { print_error(); return; }

    /* pids de externos para esperar al final */
    pid_t pids[128];
    int npids = 0;

    for (int i = 0; i < ncmd; i++) {
        char *raw = cmds[i];
        if (!raw || raw[0] == '\0') { print_error(); continue; }

        char *outfile = NULL;
        char *cmd_part = NULL;
        if (parse_redirection(raw, &cmd_part, &outfile) != 0) {
            if (outfile) free(outfile);
            print_error();
            continue;
        }

        /* tokenizar comando */
        char *argv[128];
        int argc = tokenize(cmd_part, argv, 128);
        if (argc <= 0) { if (outfile) free(outfile); continue; }
        if (argc < 0) { if (outfile) free(outfile); print_error(); continue; }

        if (is_builtin(argv[0])) {
            /* Nota: no se evalúa redirección en built-ins */
            if (outfile) {
                free(outfile);
                print_error();
                continue;
            }
            if (run_builtin(argv, argc, pl) != 0) {
                print_error();
            }
            continue;
        }

        /* externo */
        if (pl->count == 0) {
            if (outfile) free(outfile);
            print_error();
            continue;
        }

        char *exec_path = find_executable(pl, argv[0]);
        if (!exec_path) {
            if (outfile) free(outfile);
            print_error();
            continue;
        }

        pid_t pid = fork();
        if (pid < 0) {
            free(exec_path);
            if (outfile) free(outfile);
            print_error();
```

```
            continue;
        }

        if (pid == 0) {
            /* hijo */
            if (outfile) {
                if (redirect_output(outfile) != 0) {
                    print_error();
                    _exit(1);
                }
            }
            execv(exec_path, argv);
            /* si execv retorna, hay error */
            print_error();
            _exit(1);
        }

        /* padre */
        if (npids < 128) pids[npids++] = pid;
        free(exec_path);
        if (outfile) free(outfile);
    }

    /* esperar por todos los externos lanzados en esta línea */
    for (int i = 0; i < npids; i++) {
        waitpid(pids[i], NULL, 0);
    }
}

/* -------------------- main -------------------- */

int main(int argc, char *argv[]) {
    FILE *input = NULL;
    int interactive = 1;

    if (argc == 1) {
        input = stdin;
        interactive = 1;
    } else if (argc == 2) {
        interactive = 0;
        input = fopen(argv[1], "r");
        if (!input) {
            print_error();
            exit(1);
        }
    } else {
        print_error();
        exit(1);
    }

    PathList pl;
    path_init(&pl);
    /* path inicial: /bin */
    char *initv[] = { "path", "/bin", NULL };
    if (path_set(&pl, initv, 2) != 0) {
        print_error();
        exit(1);
    }

    char *line = NULL;
    size_t cap = 0;

    while (1) {
        if (interactive) {
            printf("wish> ");
            fflush(stdout);
        }

        ssize_t n = getline(&line, &cap, input);
        if (n == -1) {
            /* EOF => salir */
```

```
            exit(0);
        }

        /* quitar \n */
        if (n > 0 && (line[n - 1] == '\n' || line[n - 1] == '\r')) {
            line[n - 1] = '\0';
        }

        char *t = trim(line);
        if (t[0] == '\0') continue;

        execute_line(t, &pl);
    }

    /* no se alcanza */
    path_free(&pl);
    free(line);
    return 0;
}
```

B Makefile

```
CC=gcc
CFLAGS=-Wall -Wextra -Wpedantic -std=c11 -O2

all: wish

wish: src/wish.c
        $(CC) $(CFLAGS) -o wish src/wish.c

clean:
        rm -f wish *.o
```

C README base del repositorio

```
# Práctica 2 -- API de Procesos (wish: Wisconsin Shell)

| Campo | Detalle |
|---|---|
| Curso | Sistemas Operativos (UdeA) |
| Laboratorio | Práctica No. 2 -- API de Procesos |
| Entregable | Repositorio GitHub con `README.md` + carpeta de código fuente |
| Ejecutable | `wish` |

> Este repositorio implementa un shell simple que crea un proceso por comando externo y soporta modo
↪  interactivo y batch, built-ins, redirección y paralelismo. El enunciado especifica `wish>` como prompt y
↪  un único argumento opcional para batch.

---

## 0) Integrantes

> **Completar antes de entregar** (el enunciado exige nombres, correos y documentos).

| Nombre completo | Correo | Documento |
|---|---|---|
| **(Tu nombre)** | **(tu correo)** | **(tu documento)** |
| **(Compañero 1)** | **(correo)** | **(documento)** |
| *(Opcional)* Compañero 2 | (correo) | (documento) |

---

## 1) Descripción de la solución

Implementamos `wish` como un loop que:
```

```
1) lee una línea (con `getline()`),
2) parsea comandos (con `strsep()`),
3) ejecuta built-ins en el proceso del shell y externos con `fork()` + `execv()`,
4) espera terminación con `waitpid()` y repite.

**Modos**
- **Interactivo:** imprime `wish> ` (con espacio al final) y lee de `stdin`.
- **Batch:** `./wish batch.txt` lee comandos del archivo y **no imprime prompt**.

---

## 2) Funcionalidades implementadas

### 2.1 Built-in commands
El enunciado pide implementar `exit`, `cd` y `path` como comandos integrados.

- `exit`
  - Uso: `exit`
  - Si recibe argumentos: **error**.

- `cd` *(alias aceptado: `chd` por typo del enunciado)*
  - Uso: `cd <directorio>`
  - Si recibe 0 o más de 1 argumento: **error**.

- `path` *(alias aceptado: `route` por typo del enunciado)*
  - Uso: `path /bin /usr/bin ...`
  - Sobrescribe la ruta anterior. Si se deja vacío, el shell no ejecuta externos (solo built-ins).
  - Path inicial: `/bin` (por enunciado).

### 2.2 Ejecución de comandos externos
- Se resuelve el ejecutable buscando `cmd` dentro de cada directorio en `path` usando `access(dir/cmd, X_OK)`.
- Luego se ejecuta con `fork()` + `execv()`; el padre espera con `waitpid()`.

### 2.3 Redirección (`>`)
Soporta `comando args > archivo` con estas reglas:
- Redirige **stdout y stderr** al mismo archivo.
- Si existe, se sobrescribe (truncate).
- Son error: varios `>`, o más de un archivo a la derecha.

### 2.4 Comandos paralelos (`&`)
Soporta `cmd1 & cmd2 args & cmd3 ...`:
- Lanza todos los procesos y **luego espera** a que terminen antes de volver al prompt / siguiente línea.

### 2.5 Manejo de errores
Para cualquier error, se imprime **exactamente**:
```
An error has occurred
```
a `stderr`.

Errores fatales (terminan el shell con `exit(1)`):
- Invocar con más de un archivo batch.
- Batch file inválido (no se puede abrir).

---

## 3) Documentación del código (funciones)

Archivo principal: `src/wish.c`

- `print_error()`
  Imprime el único mensaje de error del proyecto en `stderr`.

- `trim()`
  Elimina espacios/tabs al inicio y final de una cadena (in-place).

- `tokenize()`
  Tokeniza un comando por espacios/tabs y arma `argv[]` para `execv()`.

- `split_parallel()`
  Parte una línea por `&` para crear una lista de comandos "paralelos".
```

- `parse_redirection()`
  Detecta `>` y valida el formato; retorna comando limpio y nombre de archivo destino.

- `path_set()`, `find_executable()`
  Gestionan el search path y resuelven el ejecutable usando `access(..., X_OK)`.

- `run_builtin()`
  Implementa `exit`, `cd/chd`, `path/route`.

- `execute_line()`
  Implementa la lógica completa: paralelismo + redirección + built-ins + externos.

---

4) Cómo compilar y ejecutar

4.1 Compilar
```bash
make
```

4.2 Ejecutar
Interactivo:
```bash
./wish
```

Batch:
```bash
./wish tests/batch_basic.txt
```

---

5) Pruebas realizadas

> Todas estas pruebas se pueden ejecutar manualmente copiando y pegando, o en modo batch con los archivos en
↪ `tests/`.

5.1 Básicas (externos)
- `ls`
- `ls -la /tmp`

5.2 Built-ins
- `path /bin /usr/bin`
- `cd /tmp` y luego `pwd`
- `exit`

5.3 Redirección
- `ls -la /tmp > output.txt`
- `ls /noexiste > output_err.txt` *(debe capturar también stderr)*

5.4 Paralelismo
- `sleep 1 & sleep 1 & sleep 1`
- `ls & pwd & whoami`

---

6) Problemas presentados y soluciones

- **Parseo de espacios múltiples / tabs**
  Se resolvió normalizando tokens con `strsep()` y descartando tokens vacíos.

- **Redirección de stdout y stderr al mismo archivo**
  Se implementó con `open(..., O_TRUNC)` + `dup2(fd, STDOUT_FILENO)` y `dup2(fd, STDERR_FILENO)`.

- **Esperar múltiples procesos en paralelo**
  Se guardan `pid`s y al final se hace `waitpid()` por cada uno.

```
## 7) Video de sustentación (10 minutos)

> Pegar aquí el link cuando lo suban:

- Video: **(link de YouTube)**

---

## 8) Manifiesto de transparencia (IA generativa)

- Se usó IA para:
  - Proponer una arquitectura base (funciones, parsing, estructura de carpetas).
  - Sugerir casos de prueba y checklist de errores.
  - Redactar el README (documentación y formato).

- No se usó IA para:
  - Ejecutar el código en su máquina.
  - Sustituir el proceso de depuración y pruebas finales.

---

## 9) Estructura del repositorio

```
.
|-- src/
| `-- wish.c
|-- tests/
| |-- batch_basic.txt
| |-- batch_parallel.txt
| `-- batch_redir.txt
|-- Makefile
`-- README.md
```
```

D Archivos de prueba batch

D.1 batch_basic.txt

```
# Comandos básicos
ls
pwd
whoami
```

D.2 batch_parallel.txt

```
# Paralelo
sleep 1 & sleep 1 & sleep 1
ls & pwd & whoami
```

D.3 batch_redir.txt

```
# Redirección
ls -la /tmp > out_ls.txt
ls /noexiste > out_err.txt
```